1 JVM 实战篇

1.1 JVM 参数

1.1.1 标准参数

不会随着 JDK 版本的变化而变化

-version
-help
-server
-cp
64 位 JDK 中只支持 server 模式

```
[root@localhost ~]# java -version
java version "1.8.0_191"
Java(TM) SE Runtime Environment (build 1.8.0_191-b12)
Java HotSpot(TM) 64-Bit Server VM (build 25.191-b12, mixed mode)
```

1.1.2 -X 参数

非标准参数，也就是在 JDK 各个版本中可能会变动

java -Xint -version 解释执行
java -Xcomp -version 第一次使用就编译成本地代码
java -Xmixed -version 混合模式，JVM 自己来决定

```
[root@localhost ~]# java -version
java version "1.8.0_191"
Java(TM) SE Runtime Environment (build 1.8.0_191-b12)
Java HotSpot(TM) 64-Bit Server VM (build 25.191-b12, mixed mode)
[root@localhost ~]# java -Xint -version
java version "1.8.0_191"
Java(TM) SE Runtime Environment (build 1.8.0_191-b12)
Java HotSpot(TM) 64-Bit Server VM (build 25.191-b12, interpreted mode)
[root@localhost ~]# java -Xcomp -version
java version "1.8.0_191"
Java(TM) SE Runtime Environment (build 1.8.0_191-b12)
Java HotSpot(TM) 64-Bit Server VM (build 25.191-b12, compiled mode)
[root@localhost ~]# java -Xmixed -version
java version "1.8.0_191"
Java(TM) SE Runtime Environment (build 1.8.0_191-b12)
Java HotSpot(TM) 64-Bit Server VM (build 25.191-b12, mixed mode)
```

1.1.2.1 解释器与编译器的关系（选讲）

在一个 Java 程序执行时，首先通过 javac 把 java 文件编译为虚拟机可以识别的 class 文件。然后由 JVM 解释器解释 class 文件中的字节码，通过 JVM 把解释结果转变为机器码执行。这是我们通常所说的解释执行。

C++这类语言都是直接把代码编译为机器码执行交给计算机执行，这是我们通常所说的编译执行。

两者的最大区别就是是否存在一个中间处理器去参与由代码到机器码的流程。

解释执行有个缺点就是每次都要解释字节码然后才生成机器码让计算机执行，解释的过程会占用很多的时间。于是 JVM 中诞生了编译器，编译器可以通过热点代码探测技术，找到运行次数最多的代码，把这些代码及时编译为机器码，在下次调用这些代码时跳过解释的步骤，直接执行编译好的机器码，以达到加速运行时间的目的。

Java 语言解释器与编译器并存的理念让 Java 程序得到了更加智能处理结果，是 Java 语言的特色，也是 Java 语言的优势。

1.1.2.2 解释器与编译器的工作模式

JVM 分为 Server 模式和 Client 模式，对应的编译器也有 C1（Client Complier）和 C2（Server Complier）两个编译器。C1 编译器注重编译的速度，C2 编译器注重编译的质量。

不论在 Server 模式还是 Client 模式，解释器只有一个。早期的 JVM 中解释器只能和 C1 或 C2 中的一个混合运作（当然也可以通过设置虚拟机参数，强制只有编译器运作或只有解释器运作）。

在 JDK1.7 中，分层编译的加入，让两个编译器和一个解释器可以共同参与工作。

1.1.3 -XX 参数

使用得最多的参数类型

非标准化参数，相对不稳定，主要用于 JVM 调优和 Debug

a.Boolean 类型
格式：-XX:[+-]<name>　　+或-表示启用或者禁用 name 属性
比如：-XX:+UseConcMarkSweepGC　表示启用 CMS 类型的垃圾回收器
　　-XX:+UseG1GC　　表示启用 G1 类型的垃圾回收器

b.非 Boolean 类型
格式：-XX:<name>=<value> 表示 name 属性的值是 value
比如：-XX:MaxGCPauseMillis=500

1.1.4 其他参数

-Xms100M 等价于-XX:InitialHeapSize=100M
-Xmx100M 等价于-XX:MaxHeapSize=100M
-Xss100k 等价于-XX:ThreadStackSize=100k

所以这块也相当于是-XX 类型的参数

1.1.5 查看参数

java -XX:+PrintFlagsFinal -version > flags.txt

```
[root@localhost bin]# java -XX:+PrintFlagsFinal -version > flags.txt
java version "1.8.0_191"
Java(TM) SE Runtime Environment (build 1.8.0_191-b12)
Java HotSpot(TM) 64-Bit Server VM (build 25.191-b12, mixed mode)
[root@localhost bin]# sz flags.txt
```

```
[Global flags]
     intx ActiveProcessorCount                       = -1                                  {product}
    uintx AdaptiveSizeDecrementScaleFactor           = 4                                   {product}
    uintx AdaptiveSizeMajorGCDecayTimeScale          = 10                                  {product}
    uintx AdaptiveSizePausePolicy                    = 0                                   {product}
    uintx AdaptiveSizePolicyCollectionCostMargin     = 50                                  {product}
    uintx AdaptiveSizePolicyInitializingSteps        = 20                                  {product}
    uintx AdaptiveSizePolicyOutputInterval           = 0                                   {product}
    uintx AdaptiveSizePolicyWeight                   = 10                                  {product}
    uintx AdaptiveSizeThroughPutPolicy               = 0                                   {product}
    uintx AdaptiveTimeWeight                         = 25                                  {product}
     bool AdjustConcurrency                          = false                               {product}
     bool AggressiveHeap                             = false                               {product}
     bool AggressiveOpts                             = false                               {product}
```

值得注意的是"="表示默认值，":="表示被用户或 JVM 修改后的值

要想查看某个进程具体参数的值，可以使用 jinfo，这块后面聊

一般要设置参数，可以先查看一下当前参数是什么，然后进行修改

1.1.6 设置参数的方式

- 开发工具中设置比如 IDEA，eclipse
- 运行 jar 包的时候:java -XX:+UseG1GC xxx.jar

- web 容器比如 tomcat，可以在脚本中的进行设置，比如 bin 目录下的 xxx.sh/catalina.sh----->JVM 参数
- 通过 jinfo 实时调整某个 java 进程的参数(参数只有被标记为 manageable 的 flags 可以被实时修改)

1.1.7 实践和单位换算

```
1Byte(字节)=8bit(位)
1KB=1024Byte(字节)
1MB=1024KB
1GB=1024MB
1TB=1024GB
1PB=1024TB
1EB=1024PB
1ZB=1024EB
1YB=1024ZB
1BB=1024YB
1NB=1024BB
1DB=1024NB
1CB=1024DB
1XB=1024CB
....
```

```java
public class UnitDemo {
    public static void main(String[] args) {
        BigInteger i = BigInteger.valueOf(2);
        System.out.println("1KB="+i.pow(10)+"B");
        System.out.println("1MB="+i.pow(20)+"B");
        System.out.println("1GB="+i.pow(30)+"B");
        System.out.println("1TB="+i.pow(40)+"B");
        System.out.println("1PB="+i.pow(50)+"B");
        System.out.println("1EB="+i.pow(50)+"B");
        System.out.println("1ZB="+i.pow(70)+"B");
        System.out.println("1YB="+i.pow(80)+"B");
        System.out.println("1BB="+i.pow(90)+"B");
        System.out.println("1NB="+i.pow(100)+"B");
        System.out.println("1DB="+i.pow(110)+"B");
        System.out.println("1CB="+i.pow(120)+"B");
        System.out.println("1XB="+i.pow(130)+"B");
    }
}
```

```
(1)设置堆内存大小和参数打印
      -Xmx100M -Xms100M -XX:+PrintFlagsFinal
(2)查询+PrintFlagsFinal 的值
      :=true
(3)查询堆内存大小 MaxHeapSize
      := 104857600
(4)换算
```

张龙出品 必属精品

```
104857600(Byte)/1024=102400(KB)
102400(KB)/1024=100(MB)
```

(5)结论

104857600 是字节单位

1.1.8 常用参数含义

参数	含义	说明
-XX:InitialHeapSize=100M	初始化堆大小	简写-Xms100M
-XX:MaxHeapSize=100M	最大堆大小	简写-Xmx100M
-XX:NewSize=20M	设置年轻代的大小	
-XX:MaxNewSize=50M	年轻代最大大小	
-XX:OldSize=50M	设置老年代大小	
-XX:MetaspaceSize=50M	设置方法区大小	
-XX:MaxMetaspaceSize=50M	方法区最大大小	
-XX:+UseParallelGC	使用 UseParallelGC	新生代，吞吐量优先
-XX:+UseParallelOldGC	使用 UseParallelOldGC	老年代，吞吐量优先
-XX:+UseConcMarkSweepGC	使用 CMS	老年代，停顿时间优先
-XX:+UseG1GC	使用 G1GC	新生代，老年代，停顿时间优先
-XX:NewRatio	新老生代的比值	比如-XX:Ratio=4，则表示新生代:老年代=1:4，也就是新生代占整个堆内存的 1/5
-XX:SurvivorRatio	两个 S 区和 Eden 区的比值	比如-XX:SurvivorRatio=8，也就是(S0+S1):Eden=2:8，也就是一个 S 占整个新生代的 1/10
-XX:+HeapDumpOnOutOfMemoryError	启动堆内存溢出打印	当 JVM 堆内存发生溢出时，也就是 OOM，自动生

咕泡出品 必属精品

		成 dump 文件
-XX:HeapDumpPath=heap.hprof	指定堆内存溢出打印目录	表示在当前目录生成一个 heap.hprof 文件
XX:+PrintGCDetails -XX:+PrintGCTimeStamps -XX:+PrintGCDateStamps Xloggc:$CATALINA_HOME/logs/gc.log	打印出 GC 日志	可以使用不同的垃圾收集器，对比查看 GC 情况
-Xss128k	设置每个线程的堆栈大小	一个进程内的线程数经验值是 3000-5000 最佳
-XX:MaxTenuringThreshold=6	提升年老代的最大临界值	默认值为 15
-XX:InitiatingHeapOccupancyPercent	启动并发 GC 周期时堆内存使用占比	G1 之类的垃圾收集器用它来触发并发 GC 周期,基于整个堆的使用率,而不只是某一代内存的使用比. 值为 0 则表示”一直执行 GC 循环”. 默认值为 45.
-XX:G1HeapWastePercent	允许的浪费堆空间的占比	默认是 10%，如果并发标记可回收的空间小于 10%,则不会触发 MixedGC。
-XX:MaxGCPauseMillis=200ms	G1 最大停顿时间	暂停时间不能太小，太小的话就会导致出现 G1 跟不上垃圾产生的速度。最终退化成 Full GC。所以对这个参数的调优是一个持续的过程，逐步调整到最佳状态。
-XX:ConcGCThreads=n	并发垃圾收集器使用的线程数量	默认值随 JVM 运行的平台不同而不同

咕泡出品 必属精品

-XX:G1MixedGCLiveThresholdPercent=65	混合垃圾回收周期中要包括的旧区域设置占用率阈值	默认占用率为65%
-XX:G1MixedGCCountTarget=8	设置标记周期完成后，对存活数据上限为G1MixedGCLIveThresholdPercent的旧区域执行混合垃圾回收的目标次数	默认8次混合垃圾回收，混合回收的目标是要控制在此目标次数以内
-XX:G1OldCSetRegionThresholdPercent=1	描述Mixed GC时，Old Region被加入到CSet中	默认情况下，G1只把10%的Old Region加入到CSet中

1.2 常用命令

1.2.1 jps

查看 java 进程

```
The jps command lists the instrumented Java HotSpot VMs on the target s
ystem.
The command is limited to reporting information on JVMs for which it ha
s the
access permissions.
```

```
[root@localhost bin]# jps
2597 Bootstrap
2616 Jps
[root@localhost bin]# jps -l
2597 org.apache.catalina.startup.Bootstrap
2653 sun.tools.jps.Jps
```

1.2.2 jinfo

(1)实时查看和调整 JVM 配置参数

```
The jinfo command prints Java configuration information for a specified
 Java process or core file or a remote debug server. The configuration
information includes Java system properties and Java Virtual Machine (J
VM) command-line flags.
```

(2)查看

品 必 属 精

jinfo -flag name PID 查看某个 java 进程的 name 属性的值

```
jinfo -flag MaxHeapSize PID jinfo -flag UseG1GC PID
```

```
[root@localhost bin]# jinfo -flag MaxHeapSize 2597
-XX:MaxHeapSize=257949696
[root@localhost bin]# jinfo -flag UseG1GC 2597
-XX:-UseG1GC
```

(3)修改

参数只有被标记为 manageable 的 flags 可以被实时修改

jinfo -flag name=value PID

jinfo -flag [+|-]name PID

jinfo -flag = PID

(4)查看曾经赋过值的一些参数

```
jinfo -flags PID
```

```
[root@localhost bin]# jinfo -flags 2597
Attaching to process ID 2597, please wait...
Debugger attached successfully.
Server compiler detected.
JVM version is 25.191-b12
Non-default VM flags: -XX:CICompilerCount=2 -XX:InitialHeapSize=16777216 -XX:+ManagementServer -XX:MaxHeapSize=257949696
 -XX:MaxNewSize=85983232 -XX:MinHeapDeltaBytes=196608 -XX:NewSize=5570560 -XX:OldSize=11206656 -XX:+UseCompressedClassPo
inters -XX:+UseCompressedOops -XX:+UseFastUnorderedTimeStamps
Command line:  -Djava.util.logging.config.file=/usr/local/tomcat/apache-tomcat-8.5.37/conf/logging.properties -Djava.uti
l.logging.manager=org.apache.juli.ClassLoaderLogManager -Djdk.tls.ephemeralDHKeySize=2048 -Dcom.sun.management.jmxremote
 -Dcom.sun.management.jmxremote.port=8999 -Dcom.sun.management.jmxremote.authenticate=false -Dcom.sun.management.jmxremo
te.ssl=false -Djava.net.preferIPv4Stack=true -Djava.rmi.server.hostname=192.168.126.128 -Djava.protocol.handler.pkgs=org
.apache.catalina.webresources -Dorg.apache.catalina.security.SecurityListener.UMASK=0027 -Dignore.endorsed.dirs= -Dcatal
ina.base=/usr/local/tomcat/apache-tomcat-8.5.37 -Dcatalina.home=/usr/local/tomcat/apache-tomcat-8.5.37 -Djava.io.tmpdir=
/usr/local/tomcat/apache-tomcat-8.5.37/temp
```

1.2.3 jstat

基本语法

jstat [-t] [-h]

参数解释：

- option 可以从下面参数中选择
  - **-class** 显示 ClassLoad 的相关信息；
  - **-compiler** 显示 JIT 编译的相关信息；
  - **-gc** 显示和 gc 相关的堆信息；
  - **-gccapacity** 显示各个代的容量以及使用情况；

咕泡出品

- **-gccause** 显示垃圾回收的相关信息（通-gcutil）,同时显示最后一次或当前正在发生的垃圾回收的诱因；
- **-gcnew** 显示新生代信息；
- **-gcnewcapacity** 显示新生代大小和使用情况；
- **-gcold** 显示老年代和永久代的信息；
- **-gcoldcapacity** 显示老年代的大小；
- **-gcpermcapacity** 显示永久代的大小；
- **-gcutil** 显示垃圾收集信息；
- **-printcompilation** 输出 JIT 编译的方法信息；

- -t 可以在打印的列加上 Timestamp 列，用于显示系统运行的时间
- -h 可以在周期性数据数据的时候，可以在指定输出多少行以后输出一次表头
- interval 执行每次的间隔时间，单位为毫秒
- count 用于指定输出多少次记录，缺省则会一直打印

(1)查看虚拟机性能统计信息

```
The jstat command displays performance statistics for an instrumented J
ava HotSpot VM. The target JVM is identified by its virtual machine ide
ntifier, or vmid option.
```

(2)查看类装载信息

`jstat -class PID 1000 10` 查看某个 `java` 进程的类装载信息，每 `1000` 毫秒输出一次，共输出 `10` 次

咕泡出品 必属精品

```
[root@localhost bin]# jstat -class 2597 1000 10
Loaded  Bytes  Unloaded  Bytes     Time
  3042  5974.3        0     0.0     3.21
  3042  5974.3        0     0.0     3.21
  3042  5974.3        0     0.0     3.21
  3042  5974.3        0     0.0     3.21
  3042  5974.3        0     0.0     3.21
  3042  5974.3        0     0.0     3.21
  3042  5974.3        0     0.0     3.21
  3042  5974.3        0     0.0     3.21
  3042  5974.3        0     0.0     3.21
  3042  5974.3        0     0.0     3.21
```

- Loaded 加载类的数量
- Bytes 加载类合计大小
- Unloaded 卸载类的数量
- Bytes 卸载类合计大小
- Time 表示加载和卸载类总共的耗时

(3)查看垃圾收集信息

```
jstat -gc PID 1000 10
```

```
[root@localhost bin]# jstat -gc 2597 1000 5
 S0C    S1C    S0U    S1U      EC       EU        OC        OU       MC     MU    CCSC   CCSU   YGC     YGCT    FGC    FGCT     GCT
896.0  896.0   84.9   0.0    7488.0   794.3    18508.0   13830.4   18688.0 17984.0 2304.0 2039.3     14    0.118   1      0.023    0.141
896.0  896.0   84.9   0.0    7488.0   794.3    18508.0   13830.4   18688.0 17984.0 2304.0 2039.3     14    0.118   1      0.023    0.141
896.0  896.0   84.9   0.0    7488.0   794.3    18508.0   13830.4   18688.0 17984.0 2304.0 2039.3     14    0.118   1      0.023    0.141
896.0  896.0   84.9   0.0    7488.0   794.3    18508.0   13830.4   18688.0 17984.0 2304.0 2039.3     14    0.118   1      0.023    0.141
896.0  896.0   84.9   0.0    7488.0   882.8    18508.0   13830.4   18688.0 17984.0 2304.0 2039.3     14    0.118   1      0.023    0.141
```

- S0C: Survivor0(幸存区 0)大小（KB）
- S1C: Survivor1(幸存区 1)1 大小（KB）
- S0U: Survivor0(幸存区 0)已使用大小（KB）
- S1U: Survivor1(幸存区 1)已使用大小（KB）
- EC : Eden（伊甸区）大小（KB）
- EU : Eden（伊甸区）已使用大小（KB）

- OC：老年代大小（KB）
- OU : 老年代已使用大小（KB）
- PC : Perm 永久代大小（KB）
- PU : Perm 永久代已使用大小（KB）
- YGC：新生代 GC 个数
- YGCT：新生代 GC 的耗时（秒）
- FGC：Full GC 次数
- FGCT：Full GC 耗时（秒）
- GCT：GC 总耗时（秒）

1.2.4 jstack

(1)查看线程堆栈信息

The jstack command prints Java stack traces of Java threads for a specified Java process, core file, or remote debug server.

(2)用法

jstack PID

```
[root@localhost bin]# jstack 2597
2019-06-09 03:18:20
Full thread dump Java HotSpot(TM) 64-Bit Server VM (25.191-b12 mixed mode):

"Attach Listener" #47 daemon prio=9 os_prio=0 tid=0x00007ff81c002000 nid=0xacf waiting on condition [0x0000000000000000]
   java.lang.Thread.State: RUNNABLE

"ajp-nio-8009-AsyncTimeout" #45 daemon prio=5 os_prio=0 tid=0x00007ff844532800 nid=0xa5c waiting on condition [0x00007ff8134f3000]
   java.lang.Thread.State: TIMED_WAITING (sleeping)
        at java.lang.Thread.sleep(Native Method)
        at org.apache.coyote.AbstractProtocol$AsyncTimeout.run(AbstractProtocol.java:1149)
        at java.lang.Thread.run(Thread.java:748)

"ajp-nio-8009-Acceptor-0" #44 daemon prio=5 os_prio=0 tid=0x00007ff844530800 nid=0xa5b runnable [0x00007ff8135f4000]
   java.lang.Thread.State: RUNNABLE
        at sun.nio.ch.ServerSocketChannelImpl.accept0(Native Method)
        at sun.nio.ch.ServerSocketChannelImpl.accept(ServerSocketChannelImpl.java:422)
        at sun.nio.ch.ServerSocketChannelImpl.accept(ServerSocketChannelImpl.java:250)
        - locked <0x00000000f61143f0> (a java.lang.Object)
        at org.apache.tomcat.util.net.NioEndpoint$Acceptor.run(NioEndpoint.java:482)
        at java.lang.Thread.run(Thread.java:748)

"ajp-nio-8009-ClientPoller-0" #43 daemon prio=5 os_prio=0 tid=0x00007ff84452e800 nid=0xa5a runnable [0x00007ff8136f5000]
   java.lang.Thread.State: RUNNABLE
        at sun.nio.ch.EPollArrayWrapper.epollWait(Native Method)
        at sun.nio.ch.EPollArrayWrapper.poll(EPollArrayWrapper.java:269)
        at sun.nio.ch.EPollSelectorImpl.doSelect(EPollSelectorImpl.java:93)
        at sun.nio.ch.SelectorImpl.lockAndDoSelect(SelectorImpl.java:86)
        - locked <0x00000000f1230e40> (a sun.nio.ch.Util$3)
        - locked <0x00000000f1230e30> (a java.util.Collections$UnmodifiableSet)
        - locked <0x00000000f1230e50> (a sun.nio.ch.EPollSelectorImpl)
        at sun.nio.ch.SelectorImpl.select(SelectorImpl.java:97)
        at org.apache.tomcat.util.net.NioEndpoint$Poller.run(NioEndpoint.java:825)
        at java.lang.Thread.run(Thread.java:748)
```

(4)排查死锁案例

- DeadLockDemo

```
//运行主类
public class DeadLockDemo {
    public static void main(String[] args) {
        DeadLock d1=new DeadLock(true);
        DeadLock d2=new DeadLock(false);
        Thread t1=new Thread(d1);
        Thread t2=new Thread(d2);
        t1.start();
        t2.start();
    }
}
//定义锁对象
class MyLock{
    public static Object obj1=new Object();
    public static Object obj2=new Object();
}
//死锁代码
class DeadLock implements Runnable{
    private boolean flag;
    DeadLock(boolean flag){
        this.flag=flag;
    }
    public void run() {
        if(flag) {
            while(true) {
                synchronized(MyLock.obj1) {
                    System.out.println(Thread.currentThread().getName()
+"----if 获得 obj1 锁");
                    synchronized(MyLock.obj2) {

                        System.out.println(Thread.currentThread().getNa
me()+"--- -if 获得 obj2 锁");
                    }
                }
            }
        }else {
            while(true){
                synchronized(MyLock.obj2) {
                    System.out.println(Thread.currentThread().getName()
+"----否则 获得 obj2 锁");
                    synchronized(MyLock.obj1) {

                        System.out.println(Thread.currentThread().getNa
me()+"--- -否则获得 obj1 锁");
                    }
                }
            }
        }
```

咕泡出品 必属精品

```
    }
}
```

- 运行结果

```
Run: DeadLockDemo
D:\install\dev\java\jdk1.8\bin\java.exe ...
Thread-0----if获得obj1锁
Thread-1----否则获得obj2锁
```

- jstack 分析

```
C:\Users\jack>jps -l
1152 org.jetbrains.jps.cmdline.Launcher
15760 org.jetbrains.kotlin.daemon.KotlinCompileDaemon
5140 com.gupao.gupaojvm.demo.DeadLockDemo
7940 org.jetbrains.idea.maven.server.RemoteMavenServer
8404 org/netbeans/Main
15192
15880 sun.tools.jps.Jps
7960 sun.tools.jconsole.JConsole

C:\Users\jack>jstack 5140
2019-06-10 17:17:41
Full thread dump Java HotSpot(TM) 64-Bit Server VM (25.191-b12 mixed mode):

"DestroyJavaVM" #13 prio=5 os_prio=0 tid=0x0000000003634000 nid=0x37c4 waiting on condition [0x0000000000000000]
   java.lang.Thread.State: RUNNABLE

"Thread-1" #12 prio=5 os_prio=0 tid=0x000000001e45b800 nid=0x1060 waiting for monitor entry [0x000000001eefe000]
   java.lang.Thread.State: BLOCKED (on object monitor)
        at com.gupao.gupaojvm.demo.DeadLock.run(DeadLockDemo.java:45)
        - waiting to lock <0x000000076b3eb910> (a java.lang.Object)
        - locked <0x000000076b3eb920> (a java.lang.Object)
        at java.lang.Thread.run(Thread.java:748)
```

把打印信息拉到最后可以发现

```
Found one Java-level deadlock:
=============================
"Thread-1":
  waiting to lock monitor 0x000000001cd9d9c8 (object 0x000000076b3eb910, a java.lang.Object),
  which is held by "Thread-0"
"Thread-0":
  waiting to lock monitor 0x000000001cda0468 (object 0x000000076b3eb920, a java.lang.Object),
  which is held by "Thread-1"

Java stack information for the threads listed above:
===================================================
"Thread-1":
        at com.gupao.gupaojvm.demo.DeadLock.run(DeadLockDemo.java:45)
        - waiting to lock <0x000000076b3eb910> (a java.lang.Object)
        - locked <0x000000076b3eb920> (a java.lang.Object)
        at java.lang.Thread.run(Thread.java:748)
"Thread-0":
        at com.gupao.gupaojvm.demo.DeadLock.run(DeadLockDemo.java:35)
        - waiting to lock <0x000000076b3eb920> (a java.lang.Object)
        - locked <0x000000076b3eb910> (a java.lang.Object)
        at java.lang.Thread.run(Thread.java:748)

Found 1 deadlock.
```

1.2.5 jmap

(1)生成堆转储快照

The jmap command prints shared object memory maps or heap memory detail s of a specified process, core file, or remote debug server.

(2)打印出堆内存相关信息

-XX:+PrintFlagsFinal -Xms300M -Xmx300M jmap -heap PID

咕泡出品 必属精

```
[root@localhost bin]# jmap -heap 2597
Attaching to process ID 2597, please wait...
Debugger attached successfully.
Server compiler detected.
JVM version is 25.191-b12

using thread-local object allocation.
Mark Sweep Compact GC

Heap Configuration:
   MinHeapFreeRatio         = 40
   MaxHeapFreeRatio         = 70
   MaxHeapSize              = 257949696 (246.0MB)
   NewSize                  = 5570560 (5.3125MB)
   MaxNewSize               = 85983232 (82.0MB)
   OldSize                  = 11206656 (10.6875MB)
   NewRatio                 = 2
   SurvivorRatio            = 8
   MetaspaceSize            = 21807104 (20.796875MB)
   CompressedClassSpaceSize = 1073741824 (1024.0MB)
   MaxMetaspaceSize         = 17592186044415 MB
   G1HeapRegionSize         = 0 (0.0MB)

Heap Usage:
New Generation (Eden + 1 Survivor Space):
```

(3)dump 出堆内存相关信息

jmap -dump:format=b,fifile=heap.hprof PID

```
jmap -dump:format=b,file=heap.hprof 44808
```

```
[root@localhost tmp]# jmap -dump:format=b,file=heap.hprof 2597
Dumping heap to /tmp/heap.hprof ...
File exists
[root@localhost tmp]# ll
total 26300
-rw-------. 1 root root 26930127 Jun  9 03:26 heap.hprof
```

(4)要是在发生堆内存溢出的时候，能自动 dump 出该文件就好了

一般在开发中，JVM 参数可以加上下面两句，这样内存溢出时，会自动 dump 出该文件

-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=heap.hprof

设置堆内存大小：-Xms20M -Xmx20M 启动，然后访问 localhost:9090/heap，使得堆内存溢出

(5)关于 dump 下来的文件

一般 dump 下来的文件可以结合工具来分析，这块后面再说。

1.3 常用工具

参数也了解了，命令也知道了，关键是用起来不是很方便，要是有图形化的界面就好了。

一定会有好事之者来做这件事情。（堆内存，GC 日志）

1.3.1 jconsole

JConsole 工具是 JDK 自带的可视化监控工具。查看 java 应用程序的运行概况、监控堆信息、永久区使用情况、类加载情况等。

命令行中输入：jconsole

1.3.2 jvisualvm

1.3.2.1 监控本地 Java 进程

可以监控本地的 java 进程的 CPU，类，线程等

1.3.2.2 监控远端 Java 进程

比如监控远端 tomcat，演示部署在阿里云服务器上的 tomcat

(1)在 visualvm 中选中“远程”，右击“添加”

(2)主机名上写服务器的 ip 地址，比如 31.100.39.63，然后点击“确定”

(3)右击该主机“31.100.39.63”，添加“JMX”[也就是通过 JMX 技术具体监控远端服务器哪个 Java 进程]

(4)要想让服务器上的 tomcat 被连接，需要改一下 bin/catalina.sh 这个文件

注意下面的 8998 不要和服务器上其他端口冲突

```
JAVA_OPTS="$JAVA_OPTS -Dcom.sun.management.jmxremote - Djava.rmi.server.
hostname=31.100.39.63 -Dcom.sun.management.jmxremote.port=8998 -Dcom.su
n.management.jmxremote.ssl=false - Dcom.sun.management.jmxremote.authen
ticate=true - Dcom.sun.management.jmxremote.access.file=../conf/jmxremo
te.access - Dcom.sun.management.jmxremote.password.file=../conf/jmxremo
te.password"
```

(5)在`../conf` 文件中添加两个文件 jmxremote.access 和 jmxremote.password

jmxremote.access 文件

```
guest readonly manager readwrite
```

jmxremote.password 文件

```
guest guest manager manager
```

阿里出品 必属精品

授予权限:chmod 600 *jmxremot*

(6)将连接服务器地址改为公网 ip 地址

hostname -i 查看输出情况 172.26.225.240 172.17.0.1 vim /etc/hosts
172.26.255.240 31.100.39.63

(7)设置上述端口对应的阿里云安全策略和防火墙策略

(8)启动 tomcat，来到 bin 目录

./startup.sh

(9)查看 tomcat 启动日志以及端口监听

tail -f ../logs/catalina.out lsof -i tcp:8080

(10)查看 8998 监听情况，可以发现多开了几个端口

lsof -i:8998 得到 PID netstat -antup | grep PID

(11)在刚才的 JMX 中输入 8998 端口，并且输入用户名和密码则登录成功

端口:8998 用户名:manager 密码:manager

1.3.3 Arthas

github：https://github.com/alibaba/arthas

Arthas allows developers to troubleshoot production issues for Java applications without modifying code or restarting servers.

Arthas 是 Alibaba 开源的 Java 诊断工具，采用**命令行交互模式**，是排查 jvm 相关问题的利器。

`Arthas` 是Alibaba开源的Java诊断工具，深受开发者喜爱。

当你遇到以下类似问题而束手无策时，`Arthas` 可以帮助你解决：

0. 这个类从哪个 jar 包加载的？为什么会报各种类相关的 Exception？
1. 我改的代码为什么没有执行到？难道是我没 commit？分支搞错了？
2. 遇到问题无法在线上 debug，难道只能通过加日志再重新发布吗？
3. 线上遇到某个用户的数据处理有问题，但线上同样无法 debug，线下无法重现！
4. 是否有一个全局视角来查看系统的运行状况？
5. 有什么办法可以监控到JVM的实时运行状态？
6. 怎么快速定位应用的热点，生成火焰图？

`Arthas` 支持JDK 6+，支持Linux/Mac/Windows，采用命令行交互模式，同时提供丰富的 `Tab` 自动补全功能，进一步方便进行问题的定位和诊断。

1.3.3.1 下载安装

curl -O https://alibaba.github.io/arthas/arthas-boot.jar java -jar arthas-boot.jar # 然后可以选择一个 Java 进程

必看精品

Print usage

```
java -jar arthas-boot.jar -h
```

1.3.3.2 常用命令

具体每个命令怎么使用，大家可以自己查阅资料

`version`:查看 `arthas` 版本号 `help`:查看命名帮助信息 `cls`:清空屏幕 `session`:查看当前会话信息 `quit`:退出 `arthas` 客户端 `---` `dashboard`:当前进程的实时数据面板 `thread`:当前 `JVM` 的线程堆栈信息 `jvm`:查看当前 `JVM` 的信息 `sysprop`:查看 `JVM` 的系统属性 `---` `sc`:查看 `JVM` 已经加载的类信息 `dump`:`dump` 已经加载类的 `byte code` 到特定目录 `jad`:反编译指定已加载类的源码 `---` `monitor`:方法执行监控 `watch`:方法执行数据观测 `trace`:方法内部调用路径，并输出方法路径上的每个节点上耗时 `stack`:输出当前方法被调用的调用路径 `.....`

1.3.4 MAT

Java 堆分析器，用于查找内存泄漏

Heap Dump，称为堆转储文件，是 Java 进程在某个时间内的快照

下载地址：https://www.eclipse.org/mat/downloads.php

1.3.4.1 Dump 信息包含的内容

- All Objects

Class, fifields, primitive values and references

- All Classes

Classloader, name, super class, static fifields

- Garbage Collection Roots

Objects defifined to be reachable by the JVM

- Thread Stacks and Local Variables

The call-stacks of threads at the moment of the snapshot, and per-frame information about local objects

1.3.4.2 获取 Dump 文件

- 手动

```
jmap -dump:format=b,file=heap.hprof 44808
```

- 自动

```
-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=heap.hprof
```

1.3.4.3 使用

- Histogram

咕泡出品 必属精品

Histogram 可以列出内存中的对象，对象的个数及其大小

Class Name:类名称，java 类名 Objects:类的对象的数量，这个对象被创建了多少个 Shallow Heap:一个对象内存的消耗大小，不包含对其他对象的引用 Retained Heap:是 shallow Heap 的总和，即该对象被 GC 之后所能回收到内存的总和

右击类名--->List Objects--->with incoming references--->列出该类的实例

右击 Java 对象名--->Merge Shortest Paths to GC Roots--->exclude all ...--->找到 GC Root 以及原因

- Leak Suspects

查找并分析内存泄漏的可能原因

Reports--->Leak Suspects--->Details

- Top Consumers

列出大对象

1.3.5 GC 日志分析工具

要想分析日志的信息，得先拿到 GC 日志文件才行，所以得先配置一下

根据前面参数的学习，下面的配置很容易看懂

```
-XX:+PrintGCDetails -XX:+PrintGCTimeStamps -XX:+PrintGCDateStamps -Xlog gc:gc.log
```

- 在线

http://gceasy.io

- GCViewer

  java -jar .\gcviewer-1.36-SNAPSHOT.jar

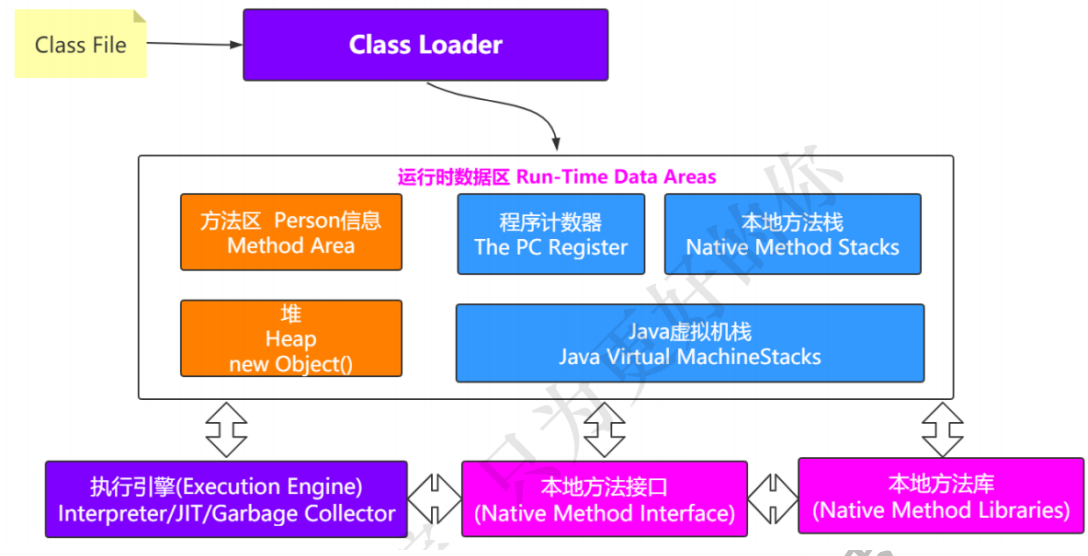
2 JVM 终结篇

2.1 重新认知 JVM

之前我们画过一张图，是从 Class 文件到类装载器，再到运行时数据区的过程。

现在咱们把这张图不妨丰富完善一下，展示了 JVM 的大体物理结构图。

路泡出品 必属精品



2.2 GC 优化

内存被使用了之后，难免会有不够用或者达到设定值的时候，就需要对内存空间进行垃圾回收。

评价一个垃圾收集器的好坏：

停顿时间 + 吞吐量 + GC 次数

2.2.1 垃圾收集发生的时机

GC 是由 JVM 自动完成的，根据 JVM 系统环境而定，所以时机是不确定的。 当然，我们可以手动进行垃圾回收，比如调用 System.gc()方法通知 JVM 进行一次垃圾回收，但是具体什么时刻运行也无法控制。也就是说 System.gc()只是通知要回收，什么时候回收由 JVM 决定。 但是不建议手动调用该方法，因为消耗的资源比较大。

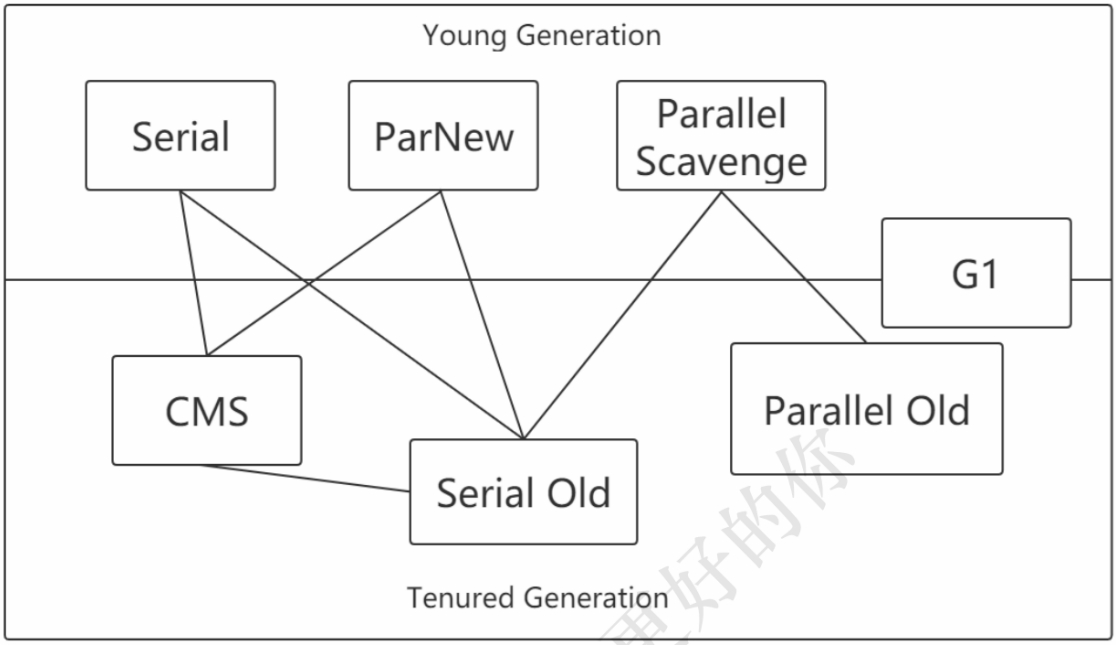
一般以下几种情况会发生垃圾回收

（1）当 Eden 区或者 S 区不够用了
（2）老年代空间不够用了
（3）方法区空间不够用了
（4）System.gc()

2.2.2 实验环境准备

比如使用 gp-jvm 这个项目，然后配置对应的参数。

2.2.3 GC 日志文件



要想分析日志的信息，得先拿到 GC 日志文件才行，所以得先配置一下，之前也看过这些参数。

```
-XX:+PrintGCDetails -XX:+PrintGCTimeStamps -XX:+PrintGCDateStamps -
Xloggc:gc.log
```

然后启动项目

可以看到默认使用的是 ParallelGC

2.2.3.1 Parallel GC 日志

【吞吐量优先】

```
2019-06-10T23:21:53.305+0800: 1.303:
[GC (Allocation Failure) [PSYoungGen: 65536K[Young 区回收前]-
>10748K[Young 区回收后]
(76288K[Young 区总大小])] 65536K[整个堆回收前]->15039K[整个堆回收
后](251392K[整个堆总大小]),
0.0113277 secs] [Times: user=0.00 sys=0.00, real=0.01 secs]
```

注意如果回收的差值中间有出入，说明这部分空间是 Old 区释放出来的

```
Java HotSpot(TM) 64-Bit Server VM (25.191-b12) for windows-amd64 JRE (1.8.0_191-b12), built on Oct  6 2018 09:29:03 by "java_re" with MS
VC++ 10.0 (VS2010)
Memory: 4k page, physical 16740520k(3394296k free), swap 21983400k(6337184k free)
CommandLine flags: -XX:InitialHeapSize=267848320 -XX:MaxHeapSize=4285573120 -XX:+PrintGC -XX:+PrintGCDateStamps -XX:+PrintGCDetails
-XX:+PrintGCTimeStamps -XX:+UseCompressedClassPointers -XX:+UseCompressedOops -XX:-UseLargePagesIndividualAllocation -XX:+UseParallelGC
2019-06-10T23:21:53.305+0800: 1.303: [GC (Allocation Failure) [PSYoungGen: 65536K->10748K(76288K)] 65536K->15039K(251392K), 0.0113277 secs]
[Times: user=0.00 sys=0.00, real=0.01 secs]
2019-06-10T23:21:53.722+0800: 1.720: [GC (Allocation Failure) [PSYoungGen: 76284K->10732K(76288K)] 80575K->22411K(251392K), 0.0099753 secs]
[Times: user=0.06 sys=0.00, real=0.01 secs]
2019-06-10T23:21:54.035+0800: 2.032: [GC (Allocation Failure) [PSYoungGen: 76268K->10741K(76288K)] 87947K->32670K(251392K), 0.0101745 secs]
[Times: user=0.02 sys=0.00, real=0.01 secs]
2019-06-10T23:21:54.441+0800: 2.439: [GC (Allocation Failure) [PSYoungGen: 76277K->10741K(141824K)] 98206K->41677K(316928K), 0.0168927 secs]
[Times: user=0.05 sys=0.00, real=0.02 secs]
2019-06-10T23:21:55.511+0800: 3.509: [GC (Metadata GC Threshold) [PSYoungGen: 126394K->10727K(141824K)] 157330K->51226K(316928K), 0.0173470
secs] [Times: user=0.02 sys=0.02, real=0.02 secs]
2019-06-10T23:21:55.528+0800: 3.527: [Full GC (Metadata GC Threshold) [PSYoungGen: 10727K->0K(141824K)] [ParOldGen: 40499K->21331K(121856K)]
51226K->21331K(263680K), [Metaspace: 20652K->20651K(1069056K)], 0.0418448 secs] [Times: user=0.13 sys=0.00, real=0.04 secs]
2019-06-10T23:21:56.921+0800: 4.919: [GC (Allocation Failure) [PSYoungGen: 131072K->13286K(225280K)] 152403K->34625K(347136K), 0.0087310
secs] [Times: user=0.01 sys=0.00, real=0.01 secs]
2019-06-10T23:21:57.751+0800: 5.749: [GC (Metadata GC Threshold) [PSYoungGen: 74979K->7932K(274944K)] 96319K->29279K(396800K), 0.0121696
secs] [Times: user=0.03 sys=0.00, real=0.01 secs]
2019-06-10T23:21:57.763+0800: 5.761: [Full GC (Metadata GC Threshold) [PSYoungGen: 7932K->0K(274944K)] [ParOldGen: 21347K->18511K(165888K)]
29279K->18511K(440832K), [Metaspace: 34301K->34301K(1081344K)], 0.0562150 secs] [Times: user=0.09 sys=0.00, real=0.06 secs]
```

2.2.3.2 CMS 日志

【停顿时间优先】

参数设置：-XX:+UseConcMarkSweepGC -Xloggc:cms-gc.log

2.2.3.3 G1 日志

【停顿时间优先】

参数设置：-XX:+UseG1GC -Xloggc:g1-gc.log

理解 G1 日志格式：https://blogs.oracle.com/poonam/understanding-g1-gc-logs

```
-XX:+UseG1GC # 使用了 G1 垃圾收集器
# 什么时候发生的 GC，相对的时间刻，GC 发生的区域 young，总共花费的时间，
0.00478s，
# It is a stop-the-world activity and all
# the application threads are stopped at a safepoint during this time.
2019-12-18T16:06:46.508+0800: 0.458: [GC pause (G1 Evacuation Pause)
(young), 0.0047804 secs]
# 多少个垃圾回收线程，并行的时间
[Parallel Time: 3.0 ms, GC Workers: 4]
# GC 线程开始相对于上面的 0.458 的时间刻
[GC Worker Start (ms): Min: 458.5, Avg: 458.5, Max: 458.5, Diff: 0.0]
# This gives us the time spent by each worker thread scanning the roots
# (globals, registers, thread stacks and VM data structures).
[Ext Root Scanning (ms): Min: 0.2, Avg: 0.4, Max: 0.7, Diff: 0.5, Sum:
1.7]
# Update RS gives us the time each thread spent in updating the
Remembered Sets.
[Update RS (ms): Min: 0.0, Avg: 0.0, Max: 0.0, Diff: 0.0, Sum: 0.0]

...
```

咕泡出品 必属精品

```
Java HotSpot(TM) 64-Bit Server VM (25.191-b12) for windows-amd64 JRE (1.8.0_191-b12), built on Oct  6
2018 09:29:03 by "java_re" with MS VC++ 10.0 (VS2010)
Memory: 4k page, physical 16740520k(3691148k free), swap 21983400k(6706548k free)
CommandLine flags: -XX:InitialHeapSize=267848320 -XX:MaxHeapSize=4285573120 -XX:+PrintGC
-XX:+PrintGCDateStamps -XX:+PrintGCDetails -XX:+PrintGCTimeStamps -XX:+UseCompressedClassPointers
-XX:+UseCompressedOops -XX:+UseG1GC -XX:-UseLargePagesIndividualAllocation
2019-06-10T23:58:51.614+0800: 0.541: [GC pause (G1 Evacuation Pause) (young), 0.0037880 secs]
   [Parallel Time: 2.8 ms, GC Workers: 4]
      [GC Worker Start (ms): Min: 541.3, Avg: 541.3, Max: 541.3, Diff: 0.0]
      [Ext Root Scanning (ms): Min: 0.0, Avg: 0.3, Max: 0.5, Diff: 0.5, Sum: 1.3]
      [Update RS (ms): Min: 0.0, Avg: 0.0, Max: 0.0, Diff: 0.0, Sum: 0.0]
         [Processed Buffers: Min: 0, Avg: 0.0, Max: 0, Diff: 0, Sum: 0]
      [Scan RS (ms): Min: 0.0, Avg: 0.0, Max: 0.0, Diff: 0.0, Sum: 0.0]
      [Code Root Scanning (ms): Min: 0.0, Avg: 0.1, Max: 0.2, Diff: 0.2, Sum: 0.4]
      [Object Copy (ms): Min: 0.0, Avg: 1.6, Max: 2.2, Diff: 2.2, Sum: 6.4]
      [Termination (ms): Min: 0.0, Avg: 0.0, Max: 0.1, Diff: 0.1, Sum: 0.2]
         [Termination Attempts: Min: 1, Avg: 2.0, Max: 3, Diff: 2, Sum: 8]
      [GC Worker Other (ms): Min: 0.0, Avg: 0.7, Max: 2.8, Diff: 2.8, Sum: 2.9]
      [GC Worker Total (ms): Min: 2.8, Avg: 2.8, Max: 2.8, Diff: 0.0, Sum: 11.1]
      [GC Worker End (ms): Min: 544.1, Avg: 544.1, Max: 544.1, Diff: 0.0]
   [Code Root Fixup: 0.1 ms]
   [Code Root Purge: 0.0 ms]
   [Clear CT: 0.0 ms]
   [Other: 0.8 ms]
      [Choose CSet: 0.0 ms]
      [Ref Proc: 0.7 ms]
      [Ref Enq: 0.0 ms]
```

2.2.4 GC 日志文件分析工具

2.2.4.1 gceasy

官网：https://gceasy.io

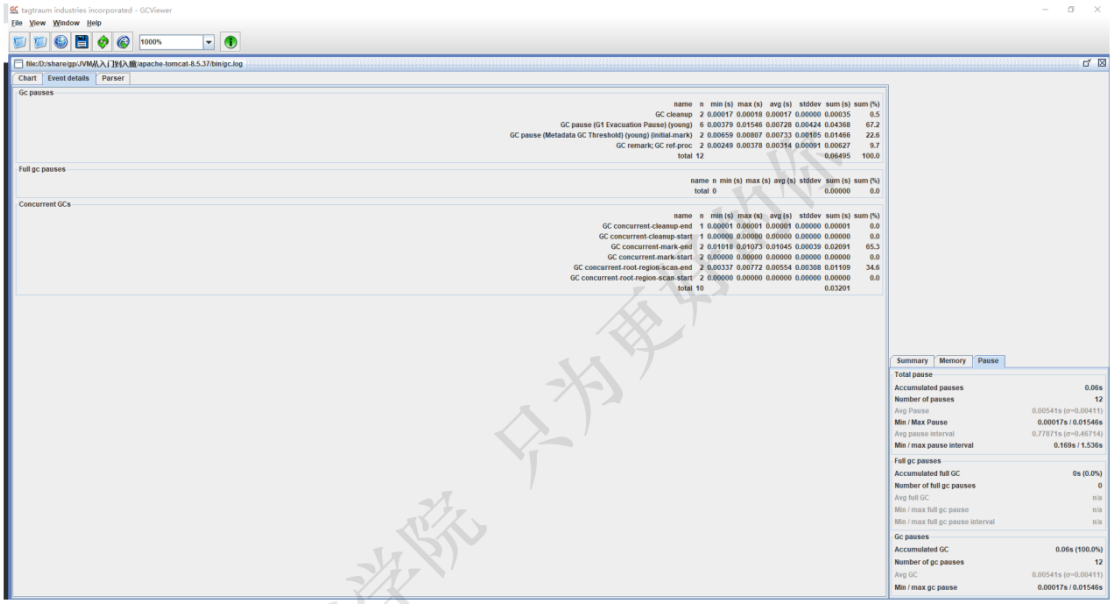
可以比较不同的垃圾收集器的吞吐量和停顿时间

比如打开 cms-gc.log 和 g1-gc.log

	Young GC	Concurrent Mark	initial-mark	Cleanup	Remark	Total
Count	6	2	2	2	2	14
Total GC Time	40.0 ms	20.9 ms	20.0 ms	0	0	80.9 ms
Avg GC Time	6.67 ms	10.5 ms	10.0 ms	0	0	5.78 ms
Avg Time std dev	4.71 ms	0.277 ms	0	0	0	5.01 ms
Min/Max Time	0 / 10.0 ms	0 / 10.7 ms	0 / 10.0 ms	0 / 0	0 / 0	0 / 10.7 ms
Avg Interval Time	940 ms	2 sec 291 ms	2 sec 284 ms	2 sec 292 ms	2 sec 291 ms	1 sec 540 ms

2.2.4.2 GCViewer

java -jar .\gcviewer-1.36-SNAPSHOT.jar



2.2.5 G1 调优与最佳指南

2.2.5.1 调优

是否选用 G1 垃圾收集器的判断依据

https://docs.oracle.com/javase/8/docs/technotes/guides/vm/G1.html#use_cases

（1）50%以上的堆被存活对象占用
（2）对象分配和晋升的速度变化非常大
（3）垃圾回收时间比较长

思考：https://blogs.oracle.com/poonam/increased-heap-usage-with-g1-gc

(1)使用 G1GC 垃圾收集器: -XX:+UseG1GC

修改配置参数，获取到 gc 日志，使用 GCViewer 分析吞吐量和响应时间

咕泡学院 只为更好的你

```
Throughput        Min Pause         Max Pause         Avg Pause
      GC count
  99.16%          0.00016s        0.0137s         0.00559s          12
```

(2)调整内存大小再获取 gc 日志分析

```
-XX:MetaspaceSize=100M
-Xms300M
-Xmx300M
```

比如设置堆内存的大小，获取到 gc 日志，使用 GCViewer 分析吞吐量和响应时间

```
Throughput        Min Pause         Max Pause         Avg Pause
      GC count
 98.89%           0.00021s            0.01531s          0.00538s
         12
```

(3)调整最大停顿时间

-XX:MaxGCPauseMillis=20 设置最大 GC 停顿时间指标

比如设置最大停顿时间，获取到 gc 日志，使用 GCViewer 分析吞吐量和响应时间

```
Throughput        Min Pause         Max Pause         Avg Pause
      GC count
 98.96%           0.00015s            0.01737s          0.00574s
          12
```

(4)启动并发 GC 时堆内存占用百分比

查看默认的占比： jinfo -flag InitiatingHeapOccupancyPercent PID-XX:InitiatingHeapOccupancyPercent=45 G1 用它来触发并发 GC 周期,基于整个堆的使用率,而不只是某一代内存的 使用比例。值为 0 则表示“一直执行 GC 循环)'. 默认值为 45 (例如, 全部的 45% 或者使用了 45%).

比如设置该百分比参数，获取到 gc 日志，使用 GCViewer 分析吞吐量和响应时间

```
Throughput        Min Pause         Max Pause         Avg Pause
      GC count
 98.11%           0.00406s      0.00532s          0.00469s            12
```

2.2.5.2 最佳指南

官网建议：
https://docs.oracle.com/javase/8/docs/technotes/guides/vm/gctuning/g1_gc_tuning.html#recommendations

(1)不要手动设置新生代和老年代的大小，只要设置整个堆的大小

G1 收集器在运行过程中，会自己调整新生代和老年代的大小 其实是通过 adapt 代的大小来调整对象晋升的速度和年龄，从而达到为收集器设置的暂停时间目标 如果手动设置了大小就意味着放弃了 G1 的自动调优

(2)不断调优暂停时间目标

咕泡出品 必属精品

一般情况下这个值设置到 100ms 或者 200ms 都是可以的(不同情况下会不一样)，但如果设置成 50ms 就不太合理。暂停 时间设置的太短，就会导致出现 G1 跟不上垃圾产生的速度。最终退化成 Full GC。所以对这个参数的调优是一个持续 的过程，逐步调整到最佳状态。暂停时间只是一个目标，并不能总是得到满足。

(3)使用-XX:ConcGCThreads=n 来增加标记线程的数量

IHOP 如果阀值设置过高，可能会遇到转移失败的风险，比如对象进行转移时空间不足。如果阀值设置过低，就会使标记周期运行过于频繁，并且有可能混合收集期回收不到空间。IHOP 值如果设置合理，但是在并发周期时间过长时，可以尝试增加并发线程数，调高 ConcGCThreads。

(4)MixedGC 调优

-XX:InitiatingHeapOccupancyPercent -XX:G1MixedGCLiveThresholdPercent -XX:G1MixedGCCountTarger -XX:G1OldCSetRegionThresholdPercent

(5)适当增加堆内存大小

2.3 JVM 性能调优的步骤

目的：对 JVM 内存的系统级的调优主要的目的是减少 GC 的频率和 Full GC 的次数。

1. Full GC
   会对整个堆进行整理，包括 Young、Tenured 和 Perm。Full GC 因为需要对整个堆进行回收，所以比较慢，因此应该尽可能减少 Full GC 的次数。

2. 导致 Full GC 的原因

   - 年老代（Tenured）被写满
     调优时尽量让对象在新生代 GC 时被回收、让对象在新生代多存活一段时间和不要创建过大的对象及数组避免直接在年老代创建对象 。

   - 持久代 Pemanet Generation 空间不足
     增大 Perm Gen 空间，避免太多静态对象 ，控制好新生代和年老代的比例

   - System.gc()被显示调用
     垃圾回收不要手动触发，尽量依靠 JVM 自身的机制
     在对 JVM 调优的过程中，很大一部分工作就是对于 FullGC 的调节，下面详细介绍对应 JVM 调优的方法和步骤。

2.3.1 JVM 性能调优方法和步骤

1. 监控 GC 的状态
   使用各种 JVM 工具，查看当前日志，分析当前 JVM 参数设置，并且分析当前堆内存快照和 gc 日志，根据实际的各区域内存划分和 GC 执行时间，觉得是否进行优化。举一个例子： 系统崩溃前的一些现象：

咕泡出品 必属精品

- 每次垃圾回收的时间越来越长，由之前的 10ms 延长到 50ms 左右，FullGC 的时间也有之前的 0.5s 延长到 4、5s
- FullGC 的次数越来越多，最频繁时隔不到 1 分钟就进行一次 FullGC
- 年老代的内存越来越大并且每次 FullGC 后年老代没有内存被释放
- 之后系统会无法响应新的请求，逐渐到达 OutOfMemoryError 的临界值，这个时候就需要分析 JVM 内存快照 dump。

2. 生成堆的 dump 文件
   - Java 的 jmap 命令来生成该文件。
3. 分析 dump 文件
   - 打开这个 3G 的堆信息文件，显然一般的 Window 系统没有这么大的内存，必须借助高配置的 Linux，几种工具打开该文件： Visual VM 、IBM HeapAnalyzer、JDK 自带的 Hprof 工具、Mat(Eclipse 专门的静态内存分析工具)推荐使用。

     注：文件太大，建议使用 Eclipse 专门的静态内存分析工具 Mat 打开分析。
4. 分析结果，判断是否需要优化
   - 如果各项参数设置合理，系统没有超时日志出现，GC 频率不高，GC 耗时不高，那么没有必要进行 GC 优化，如果 GC 时间超过 1-3 秒，或者频繁 GC，则必须优化。
     注：如果满足下面的指标，则一般不需要进行 GC：
   - Minor GC 执行时间不到 50ms；
   - Minor GC 执行不频繁，约 10 秒一次；
   - Full GC 执行时间不到 1s；
   - Full GC 执行频率不算频繁，不低于 10 分钟 1 次；
5. 调整 GC 类型和内存分配
   - 如果内存分配过大或过小，或者采用的 GC 收集器比较慢，则应该优先调整这些参数，并且先找 1 台或几台机器进行 beta，然后比较优化过的机器和没有优化的机器的性能对比，并有针对性的做出最后选择。
6. 不断分析和调整

咕泡出品 必属精品

- 通过不断的试验和试错，分析并找到最合适的参数，如果找到了最合适的参数，则将这些参数应用到所有服务器。

2.3.2 JVM 调优参数参考

1. 针对 JVM 堆的设置，一般可以通过-Xms -Xmx 限定其最小、最大值，为了防止垃圾收集器在最小、最大之间收缩堆而产生额外的时间，通常把最大、最小设置为相同的值;

2. 年轻代和年老代将根据默认的比例（1：2）分配堆内存， 可以通过调整二者之间的比率 NewRadio 来调整二者之间的大小，也可以针对回收代。比如年轻代，通过 -XX:newSize -XX:MaxNewSize 来设置其绝对大小。同样，为了防止年轻代的堆收缩，我们通常会把-XX:newSize -XX:MaxNewSize 设置为同样大小。

3. 年轻代和年老代设置多大才算合理

   - 更大的年轻代必然导致更小的年老代，大的年轻代会延长普通 GC 的周期，但会增加每次 GC 的耗时；小的年老代会导致更频繁的 Full GC

   - 更小的年轻代必然导致更大年老代，小的年轻代会导致普通 GC 很频繁，但每次的 GC 时间会更短；大的年老代会减少 Full GC 的频率
     如何选择应该依赖应用程序对象生命周期的分布情况： 如果应用存在大量的临时对象，应该选择更大的年轻代；如果存在相对较多的持久对象，年老代应该适当增大。但很多应用都没有这样明显的特性。
     在抉择时应该根据以下两点：
     （1）本着 Full GC 尽量少的原则，让年老代尽量缓存常用对象，JVM 的默认比例 1：2 也是这个道理 。
     （2）通过观察应用一段时间，看其他在峰值时年老代会占多少内存，在不影响 Full GC 的前提下，根据实际情况加大年轻代，比如可以把比例控制在 1：1。但应该给年老代至少预留 1/3 的增长空间。

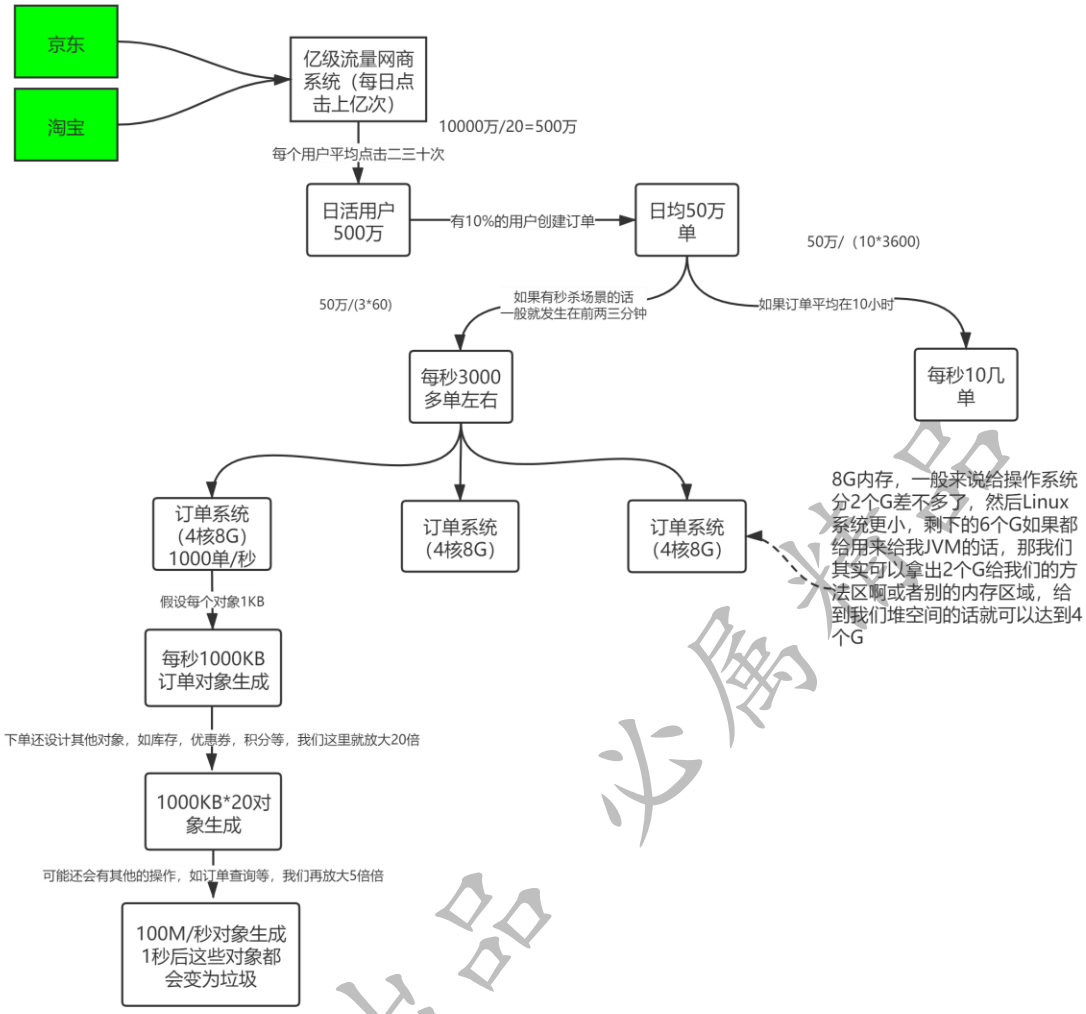
4. 在配置较好的机器上（比如多核、大内存），可以为年老代选择并行收集算法：-XX:+UseParallelOldGC 。

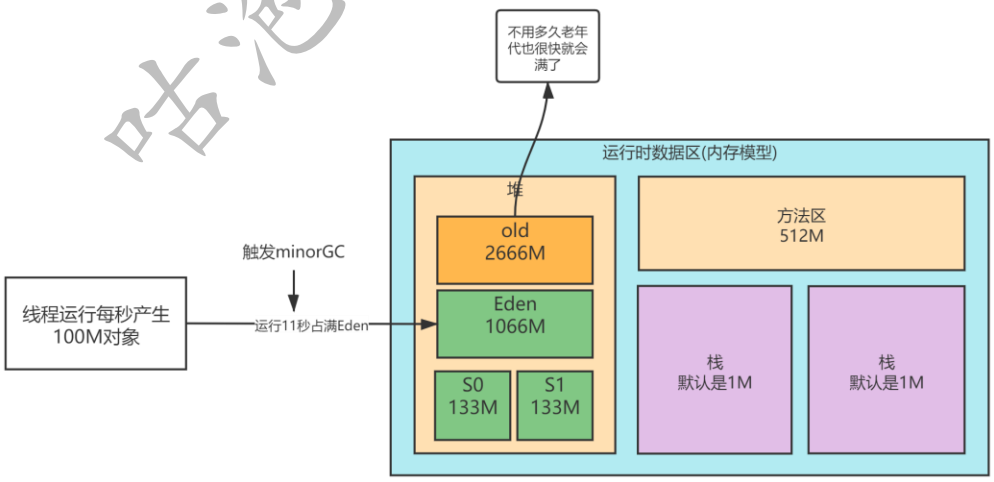
5. 线程堆栈的设置：每个线程默认会开启 1M 的堆栈，用于存放栈帧、调用参数、局部变量等，对大多数应用而言这个默认值太了，一般 256K 就足用。 理论上，在内存不变的情况下，减少每个线程的堆栈，可以产生更多的线程，但这实际上还受限于操作系统。

路漫漫 出品

2.4 高并发场景分析

咕泡出品 必属精品





可以通过-XX:NewRatio设置老年代和新生代的比例

咕泡出品 必属精品

2.5 JVM 性能优化指南

- 项目报：java.lang.OutOfMemoryError: GC overhead limit exceeded，这个异常代表：

GC 为了释放很小的空间却耗费了太多的时间，其原因一般有两个：1，堆太小，2，有死循环或大对象；

笔者首先排除了第 2 个原因，因为这个应用同时是在线上运行的，如果有问题，早就挂了。所以怀疑是这台机器中堆设置太小；

使用 ps -ef |grep "java"查看，发现：

```
-Xms768m -Xmx768m -XX:NewSize=320m -XX:MaxNewSize=320m
```

该应用的堆区设置只有 768m，而机器内存有 2g，机器上只跑这一个 java 应用，没有其他需要占用内存的地方。另外，这个应用比较大，需要占用的内存也比较多；

笔者通过上面的情况判断，只需要改变堆中各区域的大小设置即可，于是改成下面的情况：

```
er -Xms1280m -Xmx1280m -XX:NewSize=500m -XX:MaxNewSize=500m -XX
```

跟踪运行情况发现，相关异常没有再出现；

- 一个服务系统，**经常出现卡顿，分析原因，发现 Full GC 时间太长：**

jstat -gcutil:

S0 S1 E O P YGC YGCT FGC FGCT GCT

12.16 0.00 5.18 63.78 20.32 54 2.047 5 6.946 8.993

分析上面的数据，发现 Young GC 执行了 54 次，耗时 2.047 秒，每次 Young GC 耗时 37ms，在正常范围，**而 Full GC 执行了 5 次，耗时 6.946 秒，每次平均 1.389s，数据显示出来的问题是：Full GC 耗时较长**，分析该系统的是指发现，NewRatio=9，也就是说，新生代和老生代大小之比为 1:9，这就是问题的原因：

1，新生代太小，导致对象提前进入老年代，触发老年代发生 Full GC；

2，老年代较大，进行 Full GC 时耗时较大；

咕泡出品 必属精品

优化的方法是调整 NewRatio 的值，调整到 4，发现 Full GC 没有再发生，只有 Young GC 在执行。这就是把对象控制在新生代就清理掉，没有进入老年代（这种做法对一些应用是很有用的，但并不是对所有应用都要这么做）

- GC 的次数频繁怎么办？

打印出 GC 日志，到底是 minorGC 频繁还是 majorGC 频繁，结合工具看一下

（1）适当增加堆内存的空间

（2）选择垃圾收集器不合适 （如果是 G1）

（3）停顿时间是否太严格，或者堆内存的使用率可以调成高于 45%

- CPU 持续飙升

top------>哪个进程-------->jps 查看线程，jinfo 查看参数，jmap 查看堆内存

（1）集群（增加线程）

（2）mq（缓冲）

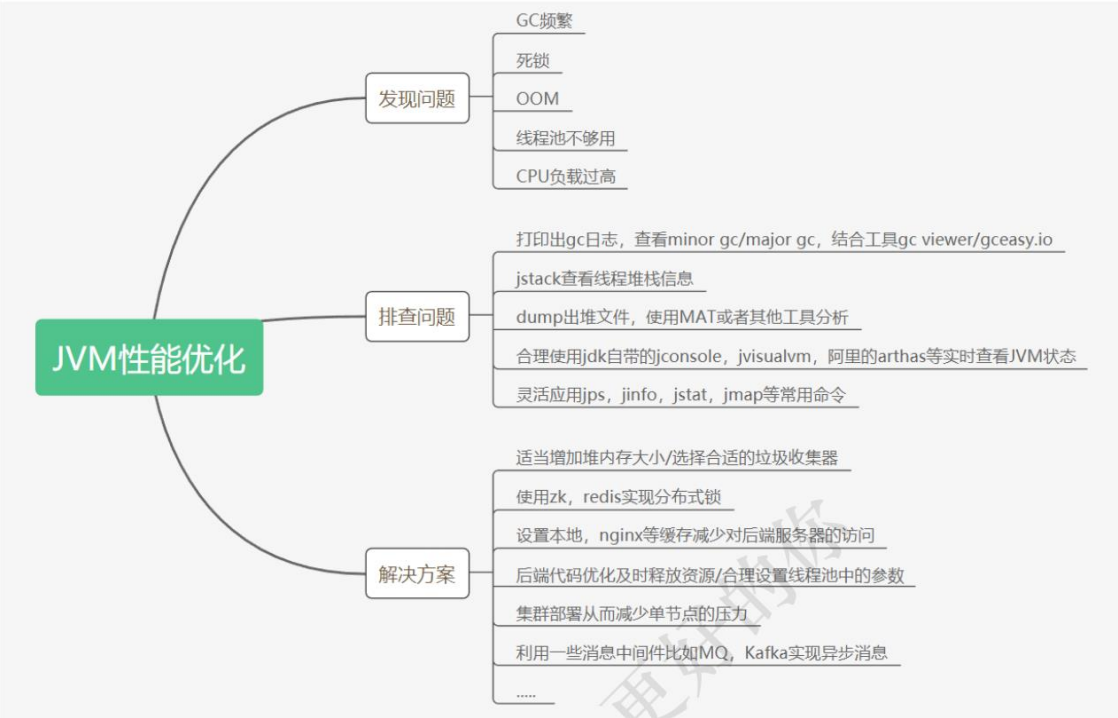
（3）业务层面是否有死锁

- OOM

看 dump 文件，日志----->分析 dump 文件---------->MAT、其他工具

（1）适当增加堆内存

咕泡出品 必属精品



2.6 常见问题思考

（1）内存泄漏与内存溢出的区别

内存泄漏：对象无法得到及时的回收，持续占用内存空间，从而造成内存空间的浪费。

内存溢出：内存泄漏到一定的程度就会导致内存溢出，但是内存溢出也有可能是大对象导致的。

（2）young gc 会有 stw 吗？

不管什么 GC，都会有 stop-the-world，只是发生时间的长短。

（3）major gc 和 full gc 的区别

major gc 指的是老年代的 gc，而 full gc 等于 young+old+metaspace 的 gc。

（4）G1 与 CMS 的区别是什么

CMS 用于老年代的回收，而 G1 用于新生代和老年代的回收。

G1 使用了 Region 方式对堆内存进行了划分，且基于标记整理算法实现，整体减少了垃圾碎片的产生。

（5）什么是直接内存

直接内存是在 java 堆外的、直接向系统申请的内存空间。通常访问直接内存的速度会优于 Java 堆。因此出于性能的考

虑，读写频繁的场合可能会考虑使用直接内存。

（6）不可达的对象一定要被回收吗？

即使在可达性分析法中不可达的对象，也并非是“非死不可”的，这时候它们暂时处于“缓刑阶段”，要真正宣告一个对象死亡，至少要经历两次标记过程；可达性分法中不可达的对象被第一次标记并且进行一次筛选，筛选的条件是此对象是否有必要执行 fifinalize 方法。当对象没有覆盖 fifinalize 方法，或 fifinalize 方法已经被虚拟机调用过时，虚拟机将这两种情况视为没有必要执行。

被判定为需要执行的对象将会被放在一个队列中进行第二次标记，除非这个对象与引用链上的任何一个对象建立关联，否则就会被真的回收。

（7）方法区中的无用类回收

方法区主要回收的是无用的类，那么如何判断一个类是无用的类的呢？

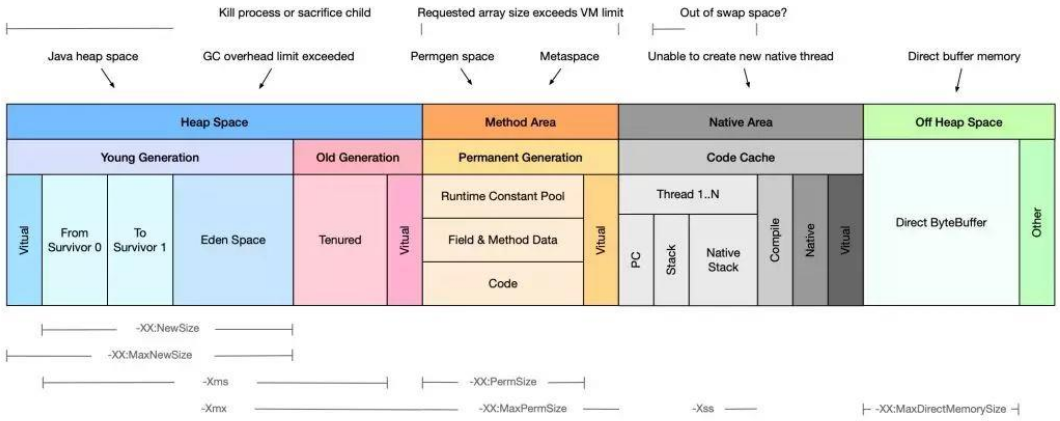
判定一个常量是否是“废弃常量”比较简单，而要判定一个类是否是“无用的类”的条件则相对苛刻许多。类需要同时满足下面 3 个条件才能算是 **“无用的类”**：

- 该类所有的实例都已经被回收，也就是 Java 堆中不存在该类的任何实例。
- 加载该类的 ClassLoader 已经被回收。
- 该类对应的 java.lang.Class 对象没有在任何地方被引用，无法在任何地方通过反射访问该类的方法。

虚拟机可以对满足上述 3 个条件的无用类进行回收，这里说的仅仅是“可以”，而并不是和对象一样不使用了就会必然被回收。

哈尔出品 必属精品

2.7 补充内容



Java 内存模型 & OOM 错误

2.7.1 heap space

当堆内存（Heap Space）没有足够空间存放新创建的对象时，就会抛出 `java.lang.OutOfMemoryError:Javaheap space` 错误（根据实际生产经验，可以对程序日志中的 OutOfMemoryError 配置关键字告警，一经发现，立即处理）。

原因分析

`heap space` 错误产生的常见原因可以分为以下几类：

1、请求创建一个超大对象，通常是一个大数组。

2、超出预期的访问量/数据量，通常是上游系统请求流量飙升，常见于各类促销/秒杀活动，可以结合业务流量指标排查是否有尖状峰值。

3、过度使用终结器（Finalizer），该对象没有立即被 GC。

4、内存泄漏（Memory Leak），大量对象引用没有释放，JVM 无法对其自动回收，常见于使用了 File 等资源没有回收。

解决方案

针对大部分情况，通常只需要通过 `-Xmx` 参数调高 JVM 堆内存空间即可。如果仍然没有解决，可以参考以下情况做进一步处理：

1、如果是超大对象，可以检查其合理性，比如是否一次性查询了数据库全部结果，而没有做结果数限制。

2、如果是业务峰值压力，可以考虑添加机器资源，或者做限流降级。

3、如果是内存泄漏，需要找到持有的对象，修改代码设计，比如关闭没有释放的连接。

2.7.2 GC overhead limit exceeded

当 Java 进程花费 98% 以上的时间执行 GC，但只恢复了不到 2% 的内存，且该动作连续重复了 5 次，就会抛出 `java.lang.OutOfMemoryError:GC overhead limit exceeded` 错误。简单地说，就是应用程序已经基本耗尽了所有可用内存， GC 也无法回收。

此类问题的原因与解决方案跟 `Javaheap space` 非常类似，可以参考上文。

2.7.3 Permgen space

该错误表示永久代（Permanent Generation）已用满，通常是因为加载的 class 数目太多或体积太大。

原因分析

永久代存储对象主要包括以下几类：

1、加载/缓存到内存中的 class 定义，包括类的名称，字段，方法和字节码；

2、常量池；

3、对象数组/类型数组所关联的 class；

4、JIT 编译器优化后的 class 信息。

PermGen 的使用量与加载到内存的 class 的数量/大小正相关。

解决方案

根据 Permgen space 报错的时机，可以采用不同的解决方案，如下所示：

1、程序启动报错，修改 `-XX:MaxPermSize` 启动参数，调大永久代空间。

2、应用重新部署时报错，很可能是没有应用没有重启，导致加载了多份 class 信息，只需重启 JVM 即可解决。

3、运行时报错，应用程序可能会动态创建大量 class，而这些 class 的生命周期很短暂，但是 JVM 默认不会卸载 class，可以设置

`-XX:+CMSClassUnloadingEnabled` 和 `-XX:+UseConcMarkSweepGC` 这两个参数允许 JVM 卸载 class。

如果上述方法无法解决，可以通过 jmap 命令 dump 内存对象 `jmap-dump:format=b,file=dump.hprof<process-id>`，然后利用 Eclipse MAT https://www.eclipse.org/mat 功能逐一分析开销最大的 classloader 和重复 class。

咕泡出品 必属精品

2.7.4 Metaspace

JDK 1.8 使用 Metaspace 替换了永久代（Permanent Generation），该错误表示 Metaspace 已被用满，通常是因为加载的 class 数目太多或体积太大。

此类问题的原因与解决方法跟 `Permgenspace` 非常类似，可以参考上文。需要特别注意的是调整 Metaspace 空间大小的启动参数为 `-XX:MaxMetaspaceSize`。

2.7.5 Unable to create new native thread

每个 Java 线程都需要占用一定的内存空间，当 JVM 向底层操作系统请求创建一个新的 native 线程时，如果没有足够的资源分配就会报此类错误。

原因分析

JVM 向 OS 请求创建 native 线程失败，就会抛出 `Unableto createnewnativethread`，常见的原因包括以下几类：

1、线程数超过操作系统最大线程数 ulimit 限制；

2、线程数超过 kernel.pid_max（只能重启）；

3、native 内存不足；

该问题发生的常见过程主要包括以下几步：

1、JVM 内部的应用程序请求创建一个新的 Java 线程；

2、JVM native 方法代理了该次请求，并向操作系统请求创建一个 native 线程；

3、操作系统尝试创建一个新的 native 线程，并为其分配内存；

4、如果操作系统的虚拟内存已耗尽，或是受到 32 位进程的地址空间限制，操作系统就会拒绝本次 native 内存分配；

5、JVM 将抛出 `java.lang.OutOfMemoryError:Unableto createnewnativethread` 错误。

解决方案

1、升级配置，为机器提供更多的内存；

2、降低 Java Heap Space 大小；

3、修复应用程序的线程泄漏问题；

4、限制线程池大小；

5、使用 -Xss 参数减少线程栈的大小；

哈泼出品 必属精品

6、调高 OS 层面的线程最大数：执行 `ulimia-a` 查看最大线程数限制，使用 `ulimit-u xxx` 调整最大线程数限制。

ulimit -a .... 省略部分内容 ..... max user processes (-u) 16384

2.7.6 Out of swap space

该错误表示所有可用的虚拟内存已被耗尽。虚拟内存（Virtual Memory）由物理内存（Physical Memory）和交换空间（Swap Space）两部分组成。当运行时程序请求的虚拟内存溢出时就会报 `Outof swap space` 错误。

原因分析

该错误出现的常见原因包括以下几类：

1、地址空间不足；

2、物理内存已耗光；

3、应用程序的本地内存泄漏（native leak），例如不断申请本地内存，却不释放。

4、执行 `jmap-histo:live<pid>` 命令，强制执行 Full GC；如果几次执行后内存明显下降，则基本确认为 Direct ByteBuffer 问题。

解决方案

根据错误原因可以采取如下解决方案：

1、升级地址空间为 64 bit；

2、使用 Arthas 检查是否为 Inflater/Deflater 解压缩问题，如果是，则显式调用 end 方法。

3、Direct ByteBuffer 问题可以通过启动参数 `-XX:MaxDirectMemorySize` 调低阈值。

4、升级服务器配置/隔离部署，避免争用。

2.7.7 Kill process or sacrifice child

有一种内核作业（Kernel Job）名为 Out of Memory Killer，它会在可用内存极低的情况下“杀死”（kill）某些进程。OOM Killer 会对所有进程进行打分，然后将评分较低的进程“杀死”，具体的评分规则可以参考 Surviving the Linux OOM Killer。

不同于其他的 OOM 错误，`Killprocessorsacrifice child` 错误不是由 JVM 层面触发的，而是由操作系统层面触发的。

原因分析

路泡出品 必属精品

默认情况下，Linux 内核允许进程申请的内存总量大于系统可用内存，通过这种“错峰复用”的方式可以更有效的利用系统资源。

然而，这种方式也会无可避免地带来一定的“超卖”风险。例如某些进程持续占用系统内存，然后导致其他进程没有可用内存。此时，系统将自动激活 OOM Killer，寻找评分低的进程，并将其“杀死”，释放内存资源。

解决方案

1、升级服务器配置/隔离部署，避免争用。

2、OOM Killer 调优。

2.7.8 Requested array size exceeds VM limit

JVM 限制了数组的最大长度，该错误表示程序请求创建的数组超过最大长度限制。

JVM 在为数组分配内存前，会检查要分配的数据结构在系统中是否可寻址，通常为 `Integer.MAX_VALUE-2`。

此类问题比较罕见，通常需要检查代码，确认业务是否需要创建如此大的数组，是否可以拆分为多个块，分批执行。

2.7.9 Direct buffer memory

Java 允许应用程序通过 Direct ByteBuffer 直接访问堆外内存，许多高性能程序通过 Direct ByteBuffer 结合内存映射文件（Memory Mapped File）实现高速 IO。

原因分析

Direct ByteBuffer 的默认大小为 64 MB，一旦使用超出限制，就会抛出 `Directbuffer memory` 错误。

解决方案

1、Java 只能通过 ByteBuffer.allocateDirect 方法使用 Direct ByteBuffer，因此，可以通过 Arthas 等在线诊断工具拦截该方法进行排查。

2、检查是否直接或间接使用了 NIO，如 netty，jetty 等。

3、通过启动参数 `-XX:MaxDirectMemorySize` 调整 Direct ByteBuffer 的上限值。

4、检查 JVM 参数是否有 `-XX:+DisableExplicitGC` 选项，如果有就去掉，因为该参数会使 `System.gc()` 失效。

5、检查堆外内存使用代码，确认是否存在内存泄漏；或者通过反射调用 `sun.misc.Cleaner` 的 `clean()` 方法来主动释放被 Direct ByteBuffer 持有的内存空间。

咕泡出品 必属精品

6、内存容量确实不足，升级配置。

咕泡出品 必屬精品